

WEALTH PULSE: SYSTEM IMPLEMENTATION PLAN

Deliverable ID: WP-IMP-03-0001

Project Title: Wealth Pulse Cross-Asset Portfolio and Educational Suite

Date: June 18, 2026

Table of Contents

WEALTH PULSE: SYSTEM IMPLEMENTATION PLAN.....	1
1. IMPLEMENTATION STRATEGY & ENVIRONMENT.....	3
1.1 Introduction.....	3
1.2 Programming Languages and Runtimes.....	3
1.3 Programming Environments and Toolchains.....	3
1.4 Testing Frameworks and Debuggers.....	4
2. REVISED IMPLEMENTATION SCHEDULE & MILESTONES.....	4
2.1 Timeline and Resource Allocation.....	4
3. CONFIGURATION MANAGEMENT & REUSE MANAGEMENT.....	5
3.1 Source Control and Versioning.....	5
3.2 Component Reuse Strategy.....	5
3.3 Build and Deployment Pipeline.....	6
4. ARCHITECTURAL & IMPLEMENTATION TRADE-OFFS.....	6

DOCUMENT METADATA

Document Ownership

Name	Position / Role	Organization	Email
Gage Radtke	Lead Software Engineer / Project Manager	Wilmington University	gradtke001@wilmu.edu
Dr. Charles Bachman	Capstone Advisor / Project Manager Reviewer	Wilmington University	cbachman@wilmu.edu

Revision History

Date	Revision	Summary of Changes	Author
06/18/2026	1.0	Initial Release: Established Dual-Stack Lifecycle, Local Baseline, and 14-Week Milestone Timeline.	G. Radtke

1. IMPLEMENTATION STRATEGY & ENVIRONMENT

1.1 Introduction

The implementation plan outlines the scheduling, resource utilization, software configurations, debugging tools, and architectural trade-offs required to transition *Wealth Pulse* from design blueprints into an operational Minimum Viable Product (MVP). By anchoring the development loop within a defined local environment, the plan minimizes external dependencies and ensures complete alignment with the constraints of a 14-week academic semester.

1.2 Programming Languages and Runtimes

The implementation leverages a modern, highly decoupled dual-stack layout:

- Backend Runtime & Language: Java 21 (Long-Term Support). Java 21 provides advanced pattern matching, virtual threads, and strong static typing, ensuring high computational integrity for asset valuation operations. The application uses the Spring Boot 3.x enterprise framework wrapper.
- Frontend Runtime & Language: ECMAScript 6+ (JavaScript) / React.js (v18+). React drives the dynamic view layers through a Single Page Application (SPA) architecture, using Vite as its optimized local compilation engine.
- Database Language: SQL (Structured Query Language), executed natively within a PostgreSQL 15+ relational database management daemon.

1.3 Programming Environments and Toolchains

To maintain a high-performance, low-latency development lifecycle, the implementation stack is entirely localized on open-source system infrastructure:

- Host Operating System: Arch Linux (Running a rolling distribution kernel), offering optimized control over background system services and database daemons.
- Hardware Profile: HP Laptop powered by an AMD Ryzen multi-core processor and 16GB of system RAM, providing rapid compile times for both the Java Virtual Machine (JVM) and Node.js environments.
- Integrated Development Environment (IDE): Code-OSS (The open-source core distribution of Visual Studio Code), customized with specialized engineering extensions:
 - *Extension Pack for Java* (Microsoft) – For automated Maven lifecycle handling, IntelliSense, and compilation warnings.
 - *Vite & React Snippets* – To accelerate frontend component creation.
 - *PostgreSQL Tools* – For executing raw SQL validation scripts directly within the development shell.

1.4 Testing Frameworks and Debuggers

Reliability is enforced through continuous validation loops across both tiers of the system using industry-standard diagnostic tools:

- Backend Debugging & Testing: Implemented using Spring Boot Developer Tools (DevTools) for live-reloading. Runtime debugging is executed using the

native Java Debugger toolset inside Code-OSS, enabling deep breakpoint analysis, thread inspection, and variable watch-states. Automated unit calculations are validated using JUnit 5 and AssertJ assertions.

- Frontend Debugging & Testing: Diagnostic monitoring utilizes Chrome Developer Tools alongside the React Developer Tools browser extension. These utilities are essential for tracking localized component state changes inside custom hooks (useAssets.js), profiling rendering latency, and monitoring outbound asynchronous HTTP network requests through the Axios network interceptor console.
- API Endpoint Validation: Postman is utilized to perform independent integration tests on REST controllers (AssetController.java) before linking them to the presentation views, ensuring correct JSON structural compliance.

2. REVISED IMPLEMENTATION SCHEDULE & MILESTONES

2.1 Timeline and Resource Allocation

The development schedule is organized around two-week Agile sprint increments across a 14-week sequence. Following the architectural decision to move all generative AI portfolio tools to a future roadmap, the mid-to-late sprint sequences have been optimized to ensure the **Asset Ledger** and the **Financial Literacy Learning Center** achieve maximum stability.

Sprint 1-2 Sprint 3-4 Sprint 5-6 Sprint 7-8 Sprint 9-10 Sprint 11-12 Sprint 13-14
[Env & DB] → [CRUD APIs] → [Live APIs] → [Dash UI] → [Learn Ctr] → [Math Eng]
→ [Testing &]

(Setup M1) (Delivery M2) (Caching M3) (Charts M4) (Modules) (Forecast M6) (Rollout M7)

Milestone	Task Description	Target Duration	Required Engineering Resources
M1	Environment Configuration, Local PostgreSQL Daemon Setup, and Relational Schema Initializations.	Days 1–14	Arch Linux, PostgreSQL Server, Code-OSS.
M2	Backend Core Delivery: Initializing Spring Boot controllers, repositories, and entities to execute basic Asset CRUD operations.	Days 15–28	Java 21, Spring Data JPA, Hibernate, Postman.
M3	Live Third-Party API Integrations: Linking external HTTPS endpoints (Alpha Vantage and GoldAPI.io) and building the database-backed caching mitigation layer.	Days 29–42	Jackson JSON Parser, Axios Client, Database Cache Tables.
M4	Front-End Layout Core: Setting up the React feature folders, integrating CSS Modules, and binding Chart.js for visualization data.	Days 43–56	React.js Framework, Vite Engine, Component-Specific CSS.
M5 (Revised)	Learning Center Execution: Developing client-side educational tools, financial glossary cards, and the interactive Options Visualizer module.	Days 57–70	Local React States, JavaScript Math Calculators.
M6	Mathematical Engine Finalization: Deploying compound interest formula scripts to map predictive retirement projections over asset class histories.	Days 71–84	Java Math Logic, Dynamic Array Transformers.

Milestone	Task Description	Target	Required Engineering
		Duration	Resources
M7	End-to-End System Testing, Security Validation	Days 85–	JUnit 5, Git, Remote
	(JWT/BCrypt audits), and Git/GitHub Remote Repository Finalization.	98	Source Control Hosting.

3. CONFIGURATION MANAGEMENT & REUSE MANAGEMENT

3.1 Source Control and Versioning

Configuration management uses **Git** initialized at the root architecture directory. Code tracking follows a strict development branching strategy to ensure main branch code remains functional:

- All feature engineering executes on isolated branches (e.g., feature/asset-ledger, feature/learning-center).
- Commits are compiled locally on the Arch Linux machine using strict Maven and Vite verification scripts before they are merged into the local repository.
- Remote synchronization is managed via an authenticated GitHub repository wrapper, ensuring a clear backup and code tracking history.

3.2 Component Reuse Strategy

To maximize development speed within the 14-week timeline, *Wealth Pulse* selectively integrates proven open-source components instead of building standard infrastructure from scratch:

- Spring Security Engine: Leveraged to provide authentication filtering, avoiding the security risks of custom-built authorization filters.

- Chart.js (via react-chartjs-2): Reused to handle canvas rendering for the portfolio pie charts and retirement forecasting graphs.
- Lombok Libraries: Reused on the backend to dynamically handle boilerplate Java getters, setters, and constructors, keeping the code clean and maintainable.

3.3 Build and Deployment Pipeline

[Code-OSS Local IDE] → [Local Compile via Maven / Vite] → [Git Commit & Push] → [GitHub Repository]

The build system relies on Apache Maven for backend dependency resolution, and Vite / npm to manage frontend JavaScript assets. Local builds are verified continuously on the HP AMD Ryzen machine to ensure compilation warnings are resolved prior to version control updates.

4. ARCHITECTURAL & IMPLEMENTATION TRADE-OFFS

The composition of this implementation plan is heavily influenced by deliberate technical trade-offs designed to balance software elegance against development constraints:

1. Feature-Driven Architecture vs. Monolithic Folder Layout:

- *Trade-off:* Dividing the React application into strict domain folders (features/auth, features/assets) creates additional folders and configuration files for a single developer.
- *Justification:* This upfront organizational cost prevents style leaking and unmaintainable files later in the project. It also provides a clean,

professional architecture that clearly demonstrates enterprise software standards to the capstone review committee.

2. Localized Component State Management vs. Global State Reducers (Redux/Zustand):

- *Trade-off:* Using local component state variables passed via custom hooks requires fresh data pulling when swapping between main views.
- *Justification:* Installing global state libraries like Redux adds complex boilerplate code that threatens the 14-week project baseline. Given the solo developer footprint and targeted MVP scope, local states significantly accelerate development speed while fully meeting user requirements.

3. Database-Backed Cache vs. Redis Enterprise Caching Layers:

- *Trade-off:* Querying a standard PostgreSQL database table to check third-party API rate limits introduces small disk read-write cycles compared to a fast, in-memory cache system like Redis.
- *Justification:* Implementing a separate Redis daemon on Arch Linux increases environment configuration complexity and local asset overhead. Utilizing existing PostgreSQL tables for tracking the 15-minute API cache window satisfies the rate limits of the free-tier data providers perfectly without introducing unnecessary tools to the local stack.

4. AI Roadmap Deferral vs. Scope Completion:

- *Trade-off:* Postponing the Spring AI prompt engine reduces the system's automated advising capabilities for the initial presentation.
- *Justification:* Removing complex large language model integrations protects the core requirements. This compromise ensures the Asset Ledger and Learning Center receive complete implementation focus, resulting in a stable application rather than an incomplete, bug-prone prototype.

Implementation Plan — Completion Addendum

Deliverable: WP-IMP-03-0001 **Revision:** 2.0 **Date:** August 3, 2026

The original pages are the planned delivery baseline. This addendum records the implemented outcome and supersedes planned tool/component names that did not become part of the repository.

Implemented Milestones

Milestone	Current outcome
M1 Environment and data	Java 21, Node 22+, PostgreSQL, Maven wrapper, Vite, and one local database named wealth_pulse .
M2 Authenticated asset API	JWT registration/login, user-isolated holdings, DTO validation, create/import/update/delete operations, and profile pictures.
M3 Pricing and cache	Stock and metal pricing, database-backed current/historical caches, scheduled refresh, fallback behavior, market tracker, and news aggregation.
M4 React dashboard	Feature-organized authenticated dashboard, asset forms/tables, stock and metal detail views, charts, goals, and responsive styles.
M5 Analytics	Transaction ledger, daily snapshots, cost basis, gain/loss, allocation, performers, benchmark comparison, and performance charts.
M6 Learning tools	Learning center content, equities/futures/stacking lessons, options calculations, and diversification tools. No AI advisory system is implemented.
M7 Verification	JUnit/Mockito tests, PostgreSQL Testcontainers integration, Vitest/Testing Library, Playwright browser tests, lint/build commands, and manual UAT checklist.

Actual Toolchain

- Backend: Spring Boot 4.1, Spring MVC, Spring Data JPA, Spring Security, JWT, PostgreSQL.
- Frontend: React 19, Vite 8, Axios, Chart.js.
- Testing: JUnit 5, Mockito, Testcontainers, Vitest, React Testing Library, Playwright.
- Documentation and commands: README.md, PROJECT_GUIDE.md, TESTING.md, and start_all.sh.

Postman and JaCoCo are not configured as repository-owned automated verification tools. They may be used manually, but the original document must not describe them as installed project automation. Styling uses feature/global CSS, not `*.module.css`.

Current Start and Verification Commands

Purpose	Command
Start both applications	<code>./start_all.sh</code>
Backend tests	<code>cd backend && ./mvnw test</code>
Frontend tests	<code>cd frontend && npm test</code>
Frontend quality/build	<code>npm run lint && npm run build</code>
Browser tests	<code>npm run test:e2e</code>

Post-Implementation Maintenance Priorities

1. Add migration-managed schema evolution instead of relying on development `ddl-auto=update`.
2. Add scheduled, tested backup/restore procedures for **wealth_pulse**.
3. Expand tests around the extracted portfolio metrics, snapshots, and report builders.
4. Measure API and UI latency before setting numerical performance service levels.
5. Keep `PROJECT_GUIDE.md` and `TESTING.md` synchronized with future structural changes.